

Insect Keypoint Inference

A research guide to running `infer_unlabeled.py`, interpreting its output, and feeding that output into Vision Transformer training

Project: c:\alignment **Model:** CPM, 6 stages, k=5 **Checkpoint:** epoch-latest.pth **Date:** 4 August 2026

1. Scope and prerequisites
2. Running the inference
3. Parameter reference
4. Reading the output files
5. What the numbers actually mean
6. Precision limits and reliability
7. Using the output to train a ViT
8. Command cookbook

1. Scope and prerequisites

This pipeline takes microscope images of soil arthropods in which *nothing has been annotated* and produces, for each specimen, a bounding box and five anatomical keypoint coordinates. It exists because the trained model cannot consume a raw camera frame directly: the network was trained exclusively on *crops* of individual specimens, so a box must be supplied at inference time. When no human has drawn one, the box has to be recovered automatically.

ENVIRONMENT

The pipeline requires the `insect_model` conda environment. The base environment does not have PyTorch, Pillow, or OpenCV installed and will fail immediately.

```
conda activate insect_model    # torch 1.12.1+cu113, CUDA available
```

All commands in this guide are written for `bash`, using `\` to continue a line. On Windows `cmd.exe` the continuation character is `^` and in PowerShell it is a backtick; paths work either way, since the script accepts forward slashes on both platforms.

VERSION COMPATIBILITY

The script runs unmodified on both the pinned environment and current releases — verified end to end on torch 2.11 / NumPy 2.4 as well as the torch 1.12 environment above. Two API changes had previously broken it outside the pin, and both are now handled at the call site: `torch.load` passes `weights_only=False` only when the installed torch accepts that keyword (PyTorch 2.6 flipped its default to `True`, which rejects the NumPy scalars in this checkpoint; the keyword did not exist before 1.13), and `gaussian` passes its shape positionally rather than as `newshape=`, which NumPy 2.1 removed.

REQUIRED INPUTS

Item	Description
<code>epoch-latest.pth</code>	Trained CPM weights (349 MB). Contains a <code>state_dict</code> key whose parameter names carry a <code>module.</code> prefix from DistributedDataParallel; the loader strips it.
<code>cpm_model_v2.py</code>	Architecture definition. Must be importable.
An input directory	Any mixture of <code>.jpg</code> <code>.jpeg</code> <code>.png</code> <code>.bmp</code> <code>.tif</code> <code>.tiff</code> . Images may be any resolution; they are not modified.

DESIGN CONSTRAINT WORTH INTERNALISING

In training (`insect_dataset.py`, lines 104–120) each image is cropped to its annotated box, resized so the longer side is 224 px, and pasted into a 224×224 canvas padded on the bottom and right. Measured across the 755 annotated images, those boxes occupy only **8%–52%** of the source frame. The model has therefore never seen a specimen that occupies less than roughly a tenth of its input. Reproducing this crop step is not an optimisation — it is a correctness requirement.

2. Running the inference

The minimal invocation, which produces annotated images and a coordinate table:

```
python infer_unlabeled.py \  
  --input data/images/inference \  
  --output output/qc \  
  --draw-box
```

The full invocation, which additionally exports everything needed for downstream ViT training (Section 7). **This is the canonical command** — it is the one to run when preparing a corpus for `pretrain.py`:

```
python infer_unlabeled.py \  
  --input      <raw_images_dir> \  
  --output     output/qc \  
  --save-crops data/crops \  
  --list-file  data/pretrain_list.txt \  
  --align-size 320 \  
  --align-fill 0.95 \  
  --align-fallback crop
```

Read that as two destinations with two very different lifetimes. `data/crops` is the *product*: the cropped-and-aligned specimen images that pre-training actually consumes, plus `data/pretrain_list.txt`, the manifest naming them. `output/qc` is *scaffolding*: the `pred_*.jpg` overlays and `keypoints.csv` you inspect to confirm the run was sane, and which you can delete afterwards.

THERE IS ONE EXPORT DIRECTORY, NOT TWO

Earlier revisions of this guide described a separate `aligned/` directory alongside `crops/`, as though cropping and aligning were two stages you could run independently. **They are not, and they never were.** The similarity transform in `align_crop` crops, rotates and rescales in a single `warpAffine` — there is no intermediate cropped-but-unaligned image for a second directory to hold. Everything `--save-crops` writes is already both cropped and aligned, so `data/crops` is the only export path you need and the only one you should point `pretrain.py` at. If you have an `output/aligned/` folder left over from an older run, it is dead weight; delete it.

2.1 How a box is obtained

Three strategies, selected with `--box-mode`:

Mode	Mechanism and when to use it
<code>auto</code> (<i>default</i>)	Segments the specimen automatically. These are microscope images in which the animal is pigmented while the background — filter paper, mesh lines, dust, scratches — is essentially colourless. The image is converted to HSV, the <i>saturation</i> channel is blurred and Otsu-thresholded, morphological opening and closing remove speckle, and the largest remaining contour is taken as the specimen. That contour's bounding rectangle is then squared up and expanded by <code>--margin</code> , because pale legs and antennae fall below the saturation threshold and would otherwise be clipped.
<code>file</code>	Reads boxes from a JSON file: <code>{"name.jpg": [x1, y1, x2, y2], ...}</code> . Use when auto-detection fails, when a frame contains several specimens, or when you have boxes from a detector or an annotation tool. Images absent from the JSON are skipped with a warning.
<code>full</code>	Uses the entire frame as the box. Correct <i>only</i> if your images are already tight specimen crops. On a raw frame this is the failure mode described in Section 6.

ASSUMPTIONS BUILT INTO AUTO MODE

Auto-detection takes the **single largest** saturated blob. It will therefore mis-handle a frame containing two specimens (it picks one, or worse, a box spanning both), and it will fail if coloured debris is larger than the animal or if the background is tinted rather than neutral. It worked on all four test frames, but it is a heuristic, not a detector. Always inspect the `pred_*.jpg` overlays with `--draw-box` before trusting a batch.

3. Parameter reference

Parameter	Default	Meaning and effect
<code>--input</code>	<code>data/images/inference</code>	Directory of images to process. Searched recursively , and any subdirectory structure — typically one folder per class — is mirrored in both <code>--output</code> and <code>--save-crops</code> . This preserves labels carried by the folder layout, and prevents a <code>001.jpg</code> in two different class folders from overwriting one another.
<code>--output</code>	<code>output/inference</code>	Destination for <code>pred_*.jpg</code> overlays and <code>keypoints.csv</code> . Created if absent. Note that one full-resolution overlay is written per input image unconditionally — useful on a test batch, but roughly 400 KB each over a large corpus.
<code>--checkpoint</code>	<code>epoch-latest.pth</code>	Trained weights. Swap this to compare training runs.
<code>--box-mode</code>	<code>auto</code>	<code>auto</code> <code>file</code> <code>full</code> . See Section 2.1.
<code>--boxes</code>	<code>boxes.json</code>	Path to the JSON box file. Only read when <code>--box-mode file</code> .
<code>--margin</code>	<code>0.15</code>	Fractional expansion of the auto-detected box. At 0.15 the box side grows by 30% total (15% each side). Raise if legs or antennae are being cut off; lower if the crop is dominated by background. This is the single most influential knob on accuracy, because it sets how much of the 224 canvas the animal fills — which is exactly the quantity the training distribution fixed.
<code>--topk</code>	<code>5</code>	Number of heatmap peaks averaged when converting a heatmap to a coordinate (Section 5.2). <code>1</code> gives a pure arg-max. Measured effect on the test images: about 10 px, i.e. negligible, because the top five cells are adjacent.
<code>--device</code>	<code>cuda</code>	Falls back to CPU automatically if CUDA is unavailable. CPU inference works but is roughly an order of magnitude slower.
<code>--draw-box</code>	<code>off</code>	Draws the crop box in green on the overlay. Strongly recommended — it is the only way to see whether auto-detection did something sensible.

EXPORT PARAMETERS (FOR DOWNSTREAM TRAINING)

Parameter	Default	Meaning and effect
<code>--save-crops</code>	<code>off</code>	The one export directory that matters. Its contents are simultaneously cropped and pose-normalised (Section 7.3): the similarity transform crops, rotates and rescales in a single operation, so there is no separate "aligned" output and no second directory to manage. Written at <code>--align-size</code> rather than 224 because MAE applies its own <code>RandomResizedCrop</code> and needs headroom to sample from.
<code>--align-pair</code>	<code>4,8</code>	The two keypoint IDs whose connecting line defines the body axis. Sets the output <i>rotation</i> . You should validate this choice against your annotation protocol — the repository stores only integer IDs, so the anatomical meaning of "keypoint 4" is not recoverable from the code.
<code>--align-size</code>	<code>256</code>	Side length of the aligned output. Keep it above 224 so <code>RandomResizedCrop(224)</code> still has room to jitter. 320 is the better choice for MAE: at <code>scale=(0.2, 1.0)</code> the low end samples about 45% linear, so a 256 px source yields a 114 px region upsampled 2× to 224 — visibly soft. From 320 that becomes 143 px, a 1.6× upsample, for roughly 1.6× the disk.
<code>--align-scale-mode</code>	<code>box</code>	What sets the zoom. <code>box</code> derives it from the detected specimen extent, so every crop shows the animal at the same apparent size; keypoints then contribute rotation only. <code>keypoints</code> is the legacy behaviour, scaling so the anchor axis spans a fixed length — see the warning in Section 7.3 before selecting it.
<code>--align-fill</code>	<code>0.95</code>	With <code>--align-scale-mode box</code> , the fraction of the output side spanned by the specimen box. At 0.95 the animal nearly fills the frame with a thin margin; lower it to include more surroundings. This is the knob that controls framing — <code>--align-span</code> no longer does.
<code>--align-span</code>	<code>0.30</code>	Fraction of the output height spanned by the anchor axis, centred. Only consulted when <code>--align-scale-mode keypoints</code> ; it has no effect under the <code>box</code> default.
<code>--align-min-score</code>	<code>0.3</code>	A specimen is treated as unalignable when either anchor keypoint scores below this. A similarity transform built on an uncertain keypoint produces a confidently wrong rotation, which is worse for training than no alignment at all.
<code>--align-fallback</code>	<code>skip</code>	What to export for unalignable specimens. <code>skip</code> omits them entirely, keeping the directory purely aligned; <code>crop</code> exports the plain bounding-box crop letterboxed to <code>--align-size</code> , trading pose consistency for a larger pre-training set. The <code>crop</code> column of <code>keypoints.csv</code> records which path each image took. Prefer <code>crop</code> for MAE pre-training — see the loss-rate warning below.
<code>--list-file</code>	<code>off</code>	Writes absolute paths of exported images, one per line — the manifest format expected by <code>ImageDatasetListV2</code> in <code>pretrain.py</code> . Only successfully exported crops are listed, so the manifest stays in sync with <code>--save-crops</code> automatically.

CHECK YOUR LOSS RATE BEFORE COMMITTING TO A LONG RUN

On the 16-image test set at `--align-min-score 0.3`, the default `--align-fallback skip` discarded **7 of 16 specimens — 44%**. That rate is a property of your imagery and model confidence, not a fixed constant, so measure it on your own corpus by counting the `crop` column of `keypoints.csv`. For MAE the trade is usually clear: reconstruction is self-supervised and tolerates imperfect pose far better than it tolerates a 44% smaller corpus, so `--align-fallback crop` is the right default there. Reserve `skip` for supervised fine-tuning, where pose consistency earns more.

4. Reading the output files

A complete run of the canonical command in Section 2 produces two trees. Given a nested input tree of class folders:

```
<raw_images_dir>/
├─ Acari/           Acari_1.jpg, 001.jpg, ...
├─ Aranae/          Aranae_1.jpg, ...
└─ Coleoptera/
   ├─ frontal/      Coleoptera_1.jpg, 001.jpg, ...
   └─ lateral/      Coleoptera_2.jpg, ...
```

you get back the *same* structure in both destinations:

```
data/                                <- the product; this is what you keep
├─ crops/                            <- cropped AND aligned, one image per specimen
│   ├─ Acari/                        Acari_1.jpg, 001.jpg, ...
│   ├─ Aranae/                      Aranae_1.jpg, ...
│   └─ Coleoptera/
│       ├─ frontal/                 Coleoptera_1.jpg, 001.jpg, ...
│       └─ lateral/                 Coleoptera_2.jpg, ...
└─ pretrain_list.txt                <- manifest of exported paths  (--list-file)

output/qc/                           <- scaffolding; safe to delete after inspection
├─ keypoints.csv                    <- the quantitative result, one row per input
├─ Acari/                          pred_Acari_1.jpg, pred_001.jpg, ...
└─ Coleoptera/frontal/              pred_Coleoptera_1.jpg, ...
```

Two consequences of the mirroring worth noting. Class labels carried by the folder layout survive the export, so `torchvision.datasets.ImageFolder` can be pointed straight at `data/crops` when you later fine-tune for classification. And because each class keeps its own subdirectory, the two distinct `001.jpg` files above remain two distinct files rather than one silently overwriting the other.

`keypoints.csv` sits at the root of the QC tree, not inside each class folder; its `filename` column carries the path relative to `--input` (e.g. `Coleoptera/frontal/001.jpg`), which is what keeps those rows unambiguous across classes.

4.1 keypoints.csv — column reference

One row per successfully processed image, 18 columns.

Column	Type	Meaning
filename	string	Path relative to <code>--input</code> , forward-slashed — <code>Coleoptera/frontal/001.jpg</code> for a nested tree, plain <code>001.jpg</code> for a flat one. Join key back to your source data, and the leading component doubles as the class label when the input is organised by class.
box	4 floats	Space-separated <code>x1 y1 x2 y2</code> of the crop box, in original-image pixels. Top-left and bottom-right corners.
crop	string	Which export path this image took under <code>--save-crops</code> : <code>aligned</code> (pose-normalised), <code>unaligned</code> (letterboxed fallback), <code>skipped</code> (no crop written), or empty when <code>--save-crops</code> was not used. Filter on this to keep a corpus purely aligned.
kp4_x, kp4_y	float	Position of keypoint 4, in original-image pixels . Origin is the top-left corner; x increases rightwards, y increases <i>downwards</i> (image convention, not Cartesian).
kp4_score	float	Confidence for keypoint 4. See Section 4.2.
kp5_* ... kp8_*	float	The same triple for keypoints 5, 6, 7 and 8.

THE COLUMNS ARE KP4–KP8, NOT KP1–KP5

Your annotation protocol has eight keypoint IDs, but `insect_dataset.py` line 147 selects channels `[3, 4, 5, 6, 7]` — zero-based — before training. **Keypoints 1, 2 and 3 were never trained and the model cannot predict them.** The column names reflect the true IDs so the CSV joins correctly against your existing annotations; do not renumber them to 1–5.

4.2 What the score means

The score is the peak value of that keypoint's predicted heatmap — the strength of the strongest response anywhere in the 28×28 grid. It is **not a calibrated probability**. It is a regression output trained with mean-squared error against Gaussian targets whose peak value is 1.0, so it behaves like a similarity-to-an-ideal-peak measure. Values slightly outside $[0, 1]$ are possible and not an error.

Interpretation in practice:

Score	Reading
> 0.6	The model found a clear, well-localised response. Usable.
$0.3 - 0.6$	Diffuse or partially occluded. Usable in aggregate, questionable per-specimen.
< 0.3	No real response. The coordinate is close to meaningless — it is the soft-arg-max of near-uniform noise, which drifts toward the centre of the map. Discard.

Always filter on score before analysis. A keypoint is emitted for every channel on every image regardless of whether the landmark is visible, so *the presence of a row does not imply a detection*.

4.3 A worked example

Two real rows from the test run, abridged:

filename	box	kp4_x	kp4_y	kp4_s	kp8_x	kp8_y	kp8_s
GRSM40_3__80.jpg	620.1 197.1 1268.8 845.9	977.12	632.92	0.650	953.88	433.58	0.993
GRSM40_2_2025-01-01_253.jpg	773.6 181.7 1149.3 557.4	990.62	310.23	0.773	819.18	449.52	0.023

Row 1 reads: in `GRSM40_3__80.jpg`, the specimen was localised to a 649×649 px box whose top-left corner is at pixel (620, 197). Keypoint 4 sits at pixel (977, 633) with moderate confidence; keypoint 8 sits at (954, 434) with very high confidence. Both coordinates are in the coordinate frame of the original 1920×1080 file, so they can be plotted on it directly with no further transformation.

Row 2 is a cautionary case. Keypoint 4 scores 0.773 and is trustworthy. Keypoint 8 scores **0.023** — the model is telling you it did not find that landmark, and the coordinate (819, 450) should be dropped, not recorded as a measurement. Note that the CSV gives you no other signal that these two numbers differ in quality; the score column is the whole mechanism.

5. What the numbers actually mean

To use these coordinates as scientific measurements you need to know the chain of transformations they came through, because each one bounds the achievable precision.

5.1 The coordinate chain

```
original image --crop--> crop --resize--> 224x224 canvas --network--> 28x28 heatmap
```

The network's native output is a 28×28 grid per keypoint. Predictions are produced in that grid and then mapped back. Forward, for the resize step:

```
scale = min( 224 / crop_width , 224 / crop_height )
```

and the inverse, applied at `infer_unlabeled.py` lines 219–221, recovers original-image pixels:

```
x_original = x_224 / scale + x1
y_original = y_224 / scale + y1
```

where $(x1, y1)$ is the box's top-left corner. Because the training pipeline pads only the bottom and right of the canvas, the origin is preserved and no padding offset enters the inverse.

5.2 From heatmap to coordinate

A heatmap cell index is converted to a sub-cell coordinate by a soft-arg-max over the `--topk` strongest cells:

```
1. flatten each 28x28 channel; take the k largest values s_j at flat indices n_j
2. recover 2-D:  c_j = n_j mod 28  (column, x)   r_j = n_j div 28  (row, y)
3. weights:      w_j = exp(s_j) / SUM_m exp(s_m)
4. estimate:     x_224 = 8 * SUM_j w_j * c_j      y_224 = 8 * SUM_j w_j * r_j
```

The factor 8 is the network stride, $224 \div 28$. One caveat on step 3: raw heatmap values lie roughly in $[0, 1]$, so the exponential compresses them and the largest achievable weight ratio is about 2.7 : 1. The softmax is therefore nearly uniform, making this closer to an unweighted mean of the top-k cells than to a peak-weighted one. On well-behaved images this is harmless because the top cells are adjacent. It matters when a channel has two competing peaks: the estimate lands between them, on neither.

5.3 Effective spatial resolution

This is the number most likely to be overlooked. Because prediction happens on a 28×28 grid, one heatmap cell corresponds to $8 / \text{scale}$ pixels of the original image — and `scale` depends on how large the box was:

Image	Box width	scale	1 cell equals	Frame area
GRSM40_2_2025-01-01_253.jpg	376 px	0.596	13.4 px	6.8%
GRSM40_2_2025-01-01_254.jpg	357 px	0.627	12.8 px	6.2%
GRSM40_3__80.jpg	649 px	0.345	23.2 px	20.3%
GRSM40_3__81.jpg	620 px	0.361	22.1 px	18.5%

So a coordinate reported as **977.12** carries about seven times fewer significant figures than the two decimal places suggest. The sub-cell interpolation of Section 5.2 buys some precision back, but **the honest resolution is on the order of 10–25 original pixels, and it varies per image with box size**. If you intend to report morphometric distances, propagate this: the same specimen photographed with a slightly different auto-box will produce slightly different numbers.

A secondary, smaller effect: both encoding (`insect_dataset.py` line 140, which floors) and decoding (multiply by 8) use the cell's top-left corner rather than its centre. This introduces a systematic bias of a few pixels toward the top-left, which largely cancels between the two directions.

6. Precision limits and reliability

Four issues that bear directly on whether a result is publishable.

6.1 Keypoints 5 and 7 are entangled

In profile views the annotation protocol collects only six points, and `insect_dataset.py` lines 93–95 fills the two missing IDs by duplication:

```
lmks[3-1] = lmks[2-1]      # kp3 := kp2  (later discarded – harmless)
lmks[7-1] = lmks[5-1]      # kp7 := kp5  (kept, and trained on)
```

A per-keypoint mask exists to suppress this — `lmk_mask`, lines 121–124 — but the dataset's `__getitem__` returns only `image`, `heatmaps`, `centermap` at line 160, so the mask is silently discarded and `train_heatmap.py` line 161 applies an unmasked MSE. The older coordinate-regression pipeline did use it (`train.py` line 155); it appears to have been dropped when the heatmap pipeline was written.

Consequence: for the 328 profile images — 43% of the 755-image training set — the model was trained to place keypoint 7 exactly on keypoint 5. The predictions are consistent with this: in the test run `kp5` and `kp7` sit about 9 px apart on one image and share an identical y coordinate on another. Treat **kp7 as unreliable**, and do not use the `kp5–kp7` distance as a morphometric feature. Fixing this requires restoring the mask and retraining.

6.2 These predictions are pseudo-labels

The four test images have no ground truth, so nothing in this pipeline has been validated against human annotation on unlabelled data. The visual overlays look correct, but "looks correct" is not a measured error. Before treating the output as data, annotate a held-out sample of perhaps 50 images by hand and compute the actual pixel error per keypoint. Until then these are model outputs, not measurements.

6.3 The box mode is not optional

To make the scale of this concrete: if a raw 1920×1080 frame is passed with `--box-mode full`, `scale` becomes 0.117 and one heatmap cell spans **69 original pixels**. A 649 px specimen would occupy just 9.5 cells of the 28-cell map. This is far outside the training distribution and the resulting coordinates are not meaningful.

6.4 Normalisation differs from the ViT pipeline

The CPM pipeline uses a bare `ToTensor()` (`insect_dataset.py` line 17) — pixels scaled to $[0, 1]$ with no mean/std normalisation. The MAE scripts in `pretrain.py` and `finetune.py` apply `Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. These are separate pipelines and do not conflict, but do not copy a transform from one into the other: feeding ImageNet-normalised tensors to the CPM will silently degrade its predictions.

7. Using the output to train a Vision Transformer

The two uploaded scripts implement the MAE recipe: `pretrain.py` performs self-supervised masked-autoencoder pre-training, and `finetune.py` adapts the resulting encoder to a classification task using the IP102 dataset. Understanding how keypoint output relates to each is mostly a matter of understanding what each script *requires*.

START HERE: WHAT THE OUTPUT CANNOT DO

Keypoint coordinates are not classification labels, and they cannot be fed to `finetune.py` as-is. That script builds `IP102(args.data_path, split='train')` and optimises `CrossEntropyLoss` (or its label-smoothed and mixup variants) against integer class indices. It expects an answer to "which species is this?". Your pipeline answers "where is landmark 4?". No amount of reformatting converts one into the other.

To fine-tune for classification you still need taxonomic labels. What the keypoint output *can* do is make that fine-tuning work better, via the paths below.

7.1 Path A — MAE pre-training on specimen crops (recommended, strongest fit)

This is the natural use, because `pretrain.py` is **self-supervised**: it masks 75% of the image patches and reconstructs them, so it consumes images and needs no labels whatsoever. Unlabelled microscope frames are exactly what it wants — and they are exactly what you have.

The contribution of the keypoint pipeline here is not the keypoints; it is the *bounding box*. Consider what happens without it. The specimen occupies 6–20% of a raw frame (Section 5.3). The training transform is:

```
transforms.RandomResizedCrop(224, scale=(0.2, 1.0), interpolation=3) # pretrain.py
```

Sampling a region covering 20%–100% of a frame in which the animal covers 6% means the overwhelming majority of training crops would contain **no specimen at all** — the model would spend its capacity learning to reconstruct filter paper and mesh lines. Pre-cropping to the detected box raises the specimen's share of every sampled view by roughly an order of magnitude.

```
# 1. Detect specimens and export cropped-and-aligned images + a manifest
python infer_unlabeled.py \
  --input      <raw_images_dir> \
  --output     output/qc \
  --save-crops data/crops \
  --list-file  data/pretrain_list.txt \
  --align-size 320 \
  --align-fill 0.95 \
  --align-fallback crop

# 2. Self-supervised pre-training on those crops
python pretrain.py \
  --data_path  data/pretrain_list.txt \
  --model      mae_vit_large_patch16 \
  --batch_size 64 \
  --mask_ratio 0.75 \
  --epochs     400 \
  --warmup_epochs 40 \
  --blr        1e-3 \
  --norm_pix_loss \
  --output_dir output_dir/mae_insect
```

Three mechanical notes. First, `--data_path` is **the manifest file itself, not a directory** — `ImageDatasetListV2` opens it and reads one path per line, which the `--list-file` export writes to match. The `/datasets01/imagenet_full_size/061417/` default in `pretrain.py` looks like a directory and misleads on this point.

Second, note that pre-training does **not** use your class folders: `ImageDatasetListV2.__getitem__` returns `(image, ref_image, 0)` — a hard-coded zero label — because MAE is self-supervised and has no use for labels. The mirrored structure in `data/crops` costs nothing here and pays off later, at the supervised fine-tuning stage, where `ImageFolder` can read those same folders as classes. Third, the learning rate is derived, not absolute:

```
lr = blr × (batch_size × accum_iter × num_gpus) / 256
```

so changing batch size without touching `--blr` automatically rescales it. Set `--lr` directly only if you want to override this.

SCALE REQUIREMENT

MAE pre-training of a ViT-Large is data-hungry: the recipe's 400 epochs and 40-epoch warm-up assume a corpus on the order of 10^5 – 10^6 images. Four crops, or four hundred, will overfit immediately and produce a worse encoder than the public ImageNet MAE checkpoint. If your unlabelled collection is small, initialise from a released MAE checkpoint and treat this as domain-adaptive continued pre-training, or use `mae_vit_base_patch16` instead of large. The crop pipeline is worth building precisely because it lets you run it over every frame you have ever collected without annotating any of them.

7.2 Path B — carrying the encoder into fine-tuning

Once pre-training has produced a checkpoint, `finetune.py` loads it. One detail in that script is worth flagging because it silently determines whether the weights actually transfer:

```
for k in checkpoint_model:
    if k.startswith('img_encoder.'):
        new_k = k.replace('img_encoder.', '')      # finetune.py
        new_state_dict[new_k] = checkpoint_model[k]
```

The loader expects encoder parameters stored under an `img_encoder.` prefix and strips it. It then calls `load_state_dict(..., strict=False)` and asserts that the only missing keys are the classifier head and `fc_norm`. If your checkpoint's parameter names do not match this convention, `strict=False` will let it load *silently with random weights* and the assertion is your only warning. Read the printed `msg` — it enumerates missing and unexpected keys, and anything beyond the four expected head parameters means the transfer did not happen.

Fine-tuning then requires labelled data, with layer-wise learning-rate decay (`--layer_decay 0.75`) so early layers move less than late ones:

```
python finetune.py \
  --finetune    output_dir/mae_insect/checkpoint-399.pth \
  --data_path   data/your_labelled_set \
  --nb_classes  <number of taxa> \
  --model       vit_large_patch16 \
  --epochs     50 \
  --blr        1e-3 \
  --layer_decay 0.75 \
  --output_dir  output_dir/ft_insect
```

Note that `--data_path` here is passed to the hard-coded `IP102` class, so pointing it at your own data requires either matching `IP102`'s directory layout or substituting your own `Dataset`.

7.3 Path C — pose alignment (this is where the keypoints themselves earn their keep)

The keypoints enable a preprocessing step that boxes alone cannot: normalising away the specimen's rotation and scale. This is the standard trick from face recognition, and it is what the project directory name — `alignment` — refers to.

Given two predicted keypoints `p_a` and `p_b`, there is a unique similarity transform (rotation, uniform scale, translation) carrying them to two fixed canonical positions `q_a` and `q_b`:

```
theta = atan2(q_b - q_a) - atan2(p_b - p_a)

[ s·cos(theta)  -s·sin(theta)  t_x ]
M  = [ s·sin(theta)   s·cos(theta)  t_y ]
```

The rotation `theta` always comes from the anchor pair. The scale `s` is the part you choose, via `--align-scale-mode`:

```
box      s = (align_size × align_fill) / box_side    t = centre - R·box_centre
keypoints s = |q_b - q_a| / |p_b - p_a|              t = q_a - R·p_a
```

Applying `M` with `cv2.warpAffine` yields an image in which the chosen body axis is always vertical and always centred. Under the `box` default the specimen also arrives at a consistent apparent size, independent of how close the original photograph was taken.

WHY THE DEFAULT IS BOX, NOT KEYPOINTS

Deriving `s` from the anchor distance normalises `|kp4 - kp8|` to a fixed length, which sounds reasonable and **inverts the framing in practice**. Because `s` is inversely proportional to that distance, a specimen photographed *closer* has a larger anchor separation and therefore comes out *smaller*.

The anchor span is a poor proxy for body size for two independent reasons. It depends on pose and anatomy — on a lateral beetle, `kp4` and `kp8` sit at opposite ends of the body; on a dorsal mite they both sit on the *idiosoma*. And it is the noisiest quantity the model emits: a low-confidence pair collapses toward a single blob centre, shrinking the denominator and inflating the zoom.

Measured on the test set, this produced an **8.5× spread** in apparent specimen size. `Coleoptera_2` came out at 46% of frame while `GRSM40_3__80` filled 97%; `GRSM40_3__81`, whose anchors collapsed to 58 px apart, was scaled to **322%** — an unusable magnified patch of cuticle. Under `box` all nine exports land at a uniform 95%. Select `keypoints` only if you specifically need the anchor axis at a fixed length and have verified your anchor confidences are uniformly high.

The benefit is a capacity argument. A ViT trained on unaligned crops must learn rotation and scale invariance from data, spending parameters and training signal on nuisance variation. Removing that variation analytically lets the model devote itself to morphology — which is the actual signal for taxonomic discrimination. The effect is largest when labelled data is scarce, which is the usual situation in this domain.

There is no separate command for this path. Alignment is not a stage you bolt on after cropping — it is the cropping, performed by the same `warpAffine`. The canonical command from Section 2 already produces aligned output; the flags below simply spell out the alignment-related defaults it is using:

```
python infer_unlabeled.py \
  --input      <raw_images_dir> \
  --output     output/qc \
  --save-crops data/crops \
  --list-file  data/pretrain_list.txt \
  --align-pair 4,8 \
  --align-size 320 \
  --align-scale-mode box \
  --align-fill 0.95 \
  --align-min-score 0.3 \
  --align-fallback crop
```

Three cautions specific to this path:

- **Choose the anchor pair deliberately.** The default 4,8 was selected because those two points are the most widely separated in the test data, and a longer baseline makes the angle estimate more stable. It is a geometric choice, not an anatomical one — the repository does not record what the IDs mean. **Do not use kp7 as an anchor** (Section 6.1).
- **Alignment amplifies keypoint error.** An error in either anchor rotates the entire image. This is why `--align-min-score` exists and why it should not be lowered casually; in the test run it correctly rejected an image whose kp8 scored 0.023. Expect to lose some fraction of your corpus, and prefer losing it to aligning it wrongly.
- **Reconsider the augmentation.** If you have deliberately removed rotation, then `RandomHorizontalFlip` and aggressive `RandomResizedCrop` scale jitter partially reintroduce it. Consider narrowing the crop scale range, e.g. `scale=(0.6, 1.0)`, when training on aligned images.

7.4 Path D — keypoints as auxiliary supervision (requires new code)

The most ambitious option is to attach a keypoint-regression head to the ViT alongside the classification head and train both, so that landmark localisation acts as an auxiliary task shaping the shared representation. This is a well-established way to improve fine-grained classification.

It is not supported by the uploaded scripts and would require: a dataset class returning (`image`, `class_label`, `keypoints`); a regression head on the encoder output; and a combined loss $L = L_{cls} + \lambda \cdot L_{kp}$. Note also that training on CPM-predicted keypoints means distilling one model's errors into another, so the human annotations in `kp_data.pkl` would be the better supervision source where they exist. Mentioned for completeness; Paths A and C give most of the benefit for far less work.

7.5 Choosing among the paths

Path	Uses	Needs labels?	Best when
A	Boxes only	No	You have many unlabelled frames. The default starting point.
B	Path A's encoder	Yes — taxonomic	You have a labelled set and want a classifier.
C	Boxes + keypoints	No (pre-train) / Yes (fine-tune)	Specimen orientation varies widely; labelled data is scarce.
D	Keypoints as targets	Yes, plus new code	A and C are exhausted and you need further gains.

8. Command cookbook

INSPECT A NEW BATCH BEFORE TRUSTING IT

```
python infer_unlabeled.py --input data/new_batch --output output/check --draw-box
# then open output/check/pred_*.jpg and confirm the green boxes contain whole specimens
```

LEGS BEING CLIPPED — WIDEN THE BOX

```
python infer_unlabeled.py --input data/new_batch --output output/check --margin 0.30 --draw-box
```

AUTO-DETECTION FAILING — SUPPLY BOXES BY HAND

```
echo '{"img_001.jpg": [620, 197, 1269, 846]}' > boxes.json
python infer_unlabeled.py --input data/new_batch --box-mode file --boxes boxes.json --output
output/check
```

Keys may be either the bare filename or the path relative to `--input` (`Coleoptera/frontal/001.jpg`); both resolve.

BUILD AN MAE PRE-TRAINING CORPUS FROM AN ENTIRE UNLABELLED ARCHIVE

The canonical command. Output is cropped and aligned in one pass, into one directory.

```
python infer_unlabeled.py \
  --input      <raw_images_dir> \
  --output     output/qc \
  --save-crops data/crops \
  --list-file  data/pretrain_list.txt \
  --align-size 320 \
  --align-fill 0.95 \
  --align-fallback crop
```

THE SAME CORPUS, KEEPING ONLY CONFIDENTLY ALIGNED SPECIMENS

Swap the fallback. This does not produce a different *kind* of output — it produces the same cropped-and-aligned images, minus the ones whose anchors were too uncertain to trust. Check the resulting row count against `keypoints.csv` before relying on it.

```
python infer_unlabeled.py \
  --input      <raw_images_dir> \
  --output     output/qc \
  --save-crops data/crops_strict \
  --list-file  data/crops_strict_list.txt \
  --align-size 320 \
  --align-fill 0.95 \
  --align-min-score 0.5 \
  --align-fallback skip
```

FRAME THE SPECIMEN MORE LOOSELY

Lower `--align-fill` to leave more background around the animal. `--align-span` has no effect here — it applies only under `--align-scale-mode keypoints`.

```
python infer_unlabeled.py --input <raw_images_dir> --output output/qc \
  --save-crops data/crops --align-fill 0.75
```

COUNT HOW MANY SPECIMENS SURVIVED THE EXPORT

```
python -c "import pandas as pd; print(pd.read_csv('output/qc/keypoints.csv')['crop'].value_counts())"
```

FILTER THE CSV TO RELIABLE KEYPOINTS ONLY

```
import pandas as pd
df = pd.read_csv("output/qc/keypoints.csv")
for kp in ["kp4", "kp5", "kp6", "kp7", "kp8"]:
    weak = df[f"{kp}_score"] < 0.3
    df.loc[weak, [f"{kp}_x", f"{kp}_y"]] = pd.NA # blank unreliable coordinates
df = df.drop(columns=["kp7_x", "kp7_y", "kp7_score"]) # see Section 6.1
```

SUMMARY OF THE HONEST CAVEATS

- Only keypoints 4–8 exist; 1–3 were never trained.
- Keypoint 7 is contaminated by a dropped training mask — do not use it.
- Spatial resolution is 10–25 original pixels and varies per image with box size.
- Scores below 0.3 mean "not found", not "found here"; filter on them.
- Auto box detection assumes one pigmented specimen on a neutral background.
- No accuracy has been measured on unlabelled data — validate against hand annotation before publishing.
- Keypoints cannot be used as classification labels for `finetune.py`.